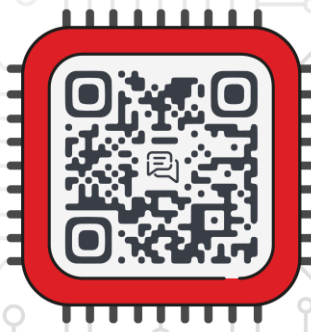




2

14

1680 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Inheritance & Polymorphism Review

عملي مشترك



RB Informatics; 7/11/2024

برمجة ٣

التمرين الأول:

قم بعمل Two classes:

Teacher class

two attributes role and collegeName

one method does() تقوم بطباعة "teaching"

ProgrammingTeacher class

يرث من الصف السابق extends teacher

and has attribute mainSubject

وضمنه main method التي سنقوم ضمنها بعمل object من class الابن و طباعة كافة المعلومات التي تخصه سواء أكانت بالأب أم به.

(ملاحظة)

الوراثة هي أحد الطرق لتمثيل علاقة is-a

ويعبر عنها بالشكل التالي :

يمكن أن نقول أن كل عنصر من نوع الابن

هو عنصر من نوع الأب كذلك.

في مثالنا السابق كل أستاذ برمجة هو

أستاذ و بالتالي تم تحقيق العلاقة

برمجياً ، يتحقق ذلك من خلال أن يرث

(يملك) الابن جميع الخصائص و التوابع

التي ليست private و ليست

constructors ضمن الأب و استخدامها

بحرية.



File: ProgrammingTeacher.java

```

1. package inheritance_concept;
2. //ProgrammingTeacher Class
3. //نستخدم extends keyword لتطبيق علاقة الوراثة كما نرى
4. public class ProgrammingTeacher extends Teacher{
5.     //attribute
6.     String mainSubject = "Programming";
7.     //main method
8.     public static void main(String args[]){
9.         //object instantiation
10.        ProgrammingTeacher programmingTeacher = new
ProgrammingTeacher();
11.        //printing the attributes
12.        //كما نرى هذه الخصائص موجودة في الأب لكن يمكن
طباعتها من كائن من نوع الابن كما أسلفنا
13.        System.out.println(programmingTeacher.collegeName);
14.        System.out.println(programmingTeacher.role);
15.        //خاصية تابعة للابن
16.        System.out.println(programmingTeacher.mainSubject);
17.        //تابع خاص بالأب لكن يمكننا استخدامه من الابن كذلك
كما نرى
18.        programmingTeacher.does();
19.    }
20. }
```

File: Teacher.java

```

1. package inheritance_concept;
2. //Teacher Class
3. class Teacher {
4.     //attributes
5.     String role = "Teacher";
6.     String collegeName = "ITE";
7.     //the method does()
8.     void does(){
9.         System.out.println("Teaching");
10.    }
11. }
```

التمرين الثاني:

عرف 5 classes :

■ الأول هو class shape ضمنه printInfo method تطبع رسالة " printing the shape info " ومن ثم الصفوف التالية:

■ Circle Class :

دعه يرث من كلاس Shape

attribute : radius (نصف القطر)

parameterized constructor

get methods for : radius-diameter -circumference -area

أي توابع لإرجاع نصف القطر والقطر و المساحة و المحيط لهذه الدائرة
then override the method printInfo()

حتى تقوم بطباعة المعلومات من توابع get التي قمنا بعملها.

■ Rectangle Class

دعه يرث من كلاس Shape

attributes : height -width

(طول و عرض)

parameterized constructor

get methods for : width-height - perimeter -area

أي توابع لإرجاع الطول و العرض و المساحة و المحيط لهذا المستطيل

then override the method printInfo()

حتى تقوم بطباعة المعلومات من توابع get التي قمنا بعملها.

■ Triangle Class

دعه يرث من كلاس Shape

attributes : height-base

(طول القاعدة و الارتفاع)

parameterized constructor

get methods for : base-height -area

أي توابع لإرجاع الطول و العرض و المساحة و لهذا المثلث

then override the method printInfo()

حتى تقوم بطباعة المعلومات من توابع get التي قمنا بعملها .

■ Main Class

فيه التابع main و الذي ضمنه سننشئ reference من shape class دون أن نحدد الكائن الذي سيشير له

(Shape shape;)

(هذا نسميه reference ، و قيمة هذا المتغير الذي يدعى shape هي null حتى نسند له الأوبجكت الذي

سيشير إليه)

ثم سنتيح للمستخدم اختيار ما هو الشكل الذي يود إدخاله (دائرة - مستطيل - مثلث)

و عند انتهائه من إدخال بيانات النوع الذي اختاره سنقوم بإسناد object من النوع الذي اختاره المستخدم

للمتغير shape الذي قمنا بعمل refernce منه من البداية ثم استدعاء التابع printInfo عليه.

ملاحظة:

Override: هو أحد شقّي تعددية الأشكال الرئيسيين ، و يعبر عن آلية يمكن من خلالها تجاوز تعريف

تابع ما موجود ضمن الصف الأب ، و عوضاً عن ذلك عمل تعريف لنفس التابع بصورة مختلفة ضمن

الصف الابن للحصول على سلوك مختلف عند مناداته من الابن.

في مثالنا السابق: لدينا معلومات مختلفة بين الأشكال التي ترث من Shape class ، لذلك نحتاج

لتعديل سلوك دالة printInfo في كل واحد من هذه الأشكال حتى تقوم بطباعة الخرج المناسب

لكل شكل ، أي سنقوم بجعل التابع يطبع معلومات الدائرة عند استدعاء التابع من هذه الدائرة

ويطبع معلومات المستطيل عند استدعاء التابع من هذا المستطيل و هكذا...

File: Shape.java

```
1. package poly_concept
2. // Shape Class
3. public class Shape {
4.     public void printInfo() {
5.         System.out.println("Printing shape information");
6.     }
7. }
```

File: Circle.java

```
1. package poly_concept
2. //Circle Class
3. public class Circle extends Shape {
4.     //attribute
5.     private double radius;
6.     //parameterized constructor
7.     public Circle(double radiusValue) {
8.         radius = radiusValue;
9.     }
10.    //ترجع نصف القطر
11.    public double getRadius() {
12.        return radius;
13.    }
14.    //ترجع القطر
15.    public double getDiameter() {
16.        return 2 * radius;
17.    }
18.    //ترجع المحيط
19.    public double getCircumference() {
20.        //Math.PI =~ 3.14
21.        return Math.PI * getDiameter();
22.    }
23.    //ترجع المساحة
24.    public double getArea() {
25.        //Math.PI =~ 3.14
26.        return Math.PI * radius * radius;
27.    }
28.
29.    //override للتابع الذي يطبع المعلومات لكي يطبع لنا معلومات الدائرة
30.    @Override
31.    public void printInfo() {
32.        System.out.println(" Radius = " + getRadius() + " , Diameter = " +
33.            getDiameter() + " , Area = " + getArea() + " , Circumference= " +
34.            getCircumference());
35.    }
36. }
```

File: Rectangle.java

```
1. package poly_concept
2. //Rectangle Class
3. public class Rectangle extends Shape {
4.     // attributes
5.     private double width;
6.     private double height;
7.     //parameterized constructor
8.     public Rectangle(double width, double height) {
9.         this.width = width;
10.        this.height = height;
11.    }
12.    // ترجع الطول
13.    public double getHeight() {
14.        return height;
15.    }
16.    // ترجع العرض
17.    public double getWidth() {
18.        return width;
19.    }
20.    // ترجع المحيط
21.    public double getPerimeter() {
22.        return 2 * (width + height);
23.    }
24.    // ترجع المساحة
25.    public double getArea() {
26.        return height * width;
27.    }
28.    // override للتابع الذي يطبع المعلومات لكي
    يطبع لنا معلومات المستطيل
29.    @Override
30.    public void printInfo() {
31.        System.out.println(" Height = " +
getHeight() + " , Width = " + getWidth() + " ,
Area = " + getArea() + " , Perimeter= " +
getPerimeter());
32.    }
33. }
```

File: Triangle.java

```
1. package poly_concept
2. // Triangle Class
3. public class Triangle extends Shape {
4.     //attributes:
5.     private double base;
6.     private double height;
7.     //parameterized constructor
8.     public Triangle(double base, double height) {
9.         this.base = base;
10.        this.height = height;
11.    }
12.
13.    // ترجع طول القاعدة
14.    public double getBase() {
15.        return base;
16.    }
17.    // ترجع الارتفاع
18.    public double getHeight() {
19.        return height;
20.    }
21.    // ترجع المساحة
22.    public double getArea() {
23.        return 0.5 * base * height;
24.    }
25.    // override للتابع الذي يطبع المعلومات لكي يطبع
    لنا معلومات المثلث
26.    @Override
27.    public void printInfo() {
28.        System.out.println(" Base = " + getBase()
+ " , Height = " + getHeight() + " , Area = " +
getArea());
29.    }
30. }
```

File: Main.java

```

1. package poly_concept
2. // including the Scanner within this java file
3. import java.util.Scanner;
4. //Main Class
5. public class Main {
6. public static void main(String[] args) {
7.     //scanner object
8.     Scanner input = new
Scanner(System.in);
9.     //shape reference
10.    Shape shape;
11.    double x, y;
12.    System.out.println(" Enter:\n 1 for
Circle \n 2 for Rectangle \n 3 for Triangle");
13.    // نختار الرقم بناء على ما نود إنشائه
14.    int choice = input.nextInt();
15.    switch (choice) {
16.        case 1:// circle
17.            System.out.print(" Enter Circle
Information : Radius = ");
18.            // إدخال نصف القطر
19.            x = input.nextDouble();
20.            //إسناد كائن من نوع دائرة
shape reference لل
21.            shape = new Circle(x);
22.            //طباعة معلومات الدائرة
23.            shape.printlnInfo();
24.            break;
25.        case 2://rectangle
26.            System.out.print(" Enter
Rectangle Information : width = ");

```

```

27.        // العرض
28.        x = input.nextDouble();
29.        System.out.print(" height = ");
30.        // الطول
31.        y = input.nextDouble();
32.        // إسناد كائن من نوع مستطيل
shape reference لل
33.        shape = new Rectangle(x, y);
34.        // طباعة معلومات المستطيل
35.        shape.printlnInfo();
36.        break;
37.        case 3:// triangle
38.            System.out.print(" Enter Triangle
Information : Base = ");
39.            // طول القاعدة
40.            x = input.nextDouble();
41.            System.out.print(" Height = ");
42.            //الارتفاع
43.            y = input.nextDouble();
44.            // إسناد كائن من نوع مثلث
shape reference لل
45.            shape = new Triangle(x, y);
46.            // طباعة معلومات المثلث
47.            shape.printlnInfo();
48.            break;
49.        default:
50.            //في حالة إدخال رقم غير 1 أو 2 أو 3
51.            System.out.println("You entered
wrong number. ");
52.        }
53.    }}

```

ملاحظة:

بما أن كل object من نوع الابن هو object من نوع الأب كذلك فيمكن التعامل مع أي object من نوع الابن على أنه من نوع الأب دون مشكلة.

بالتالي يمكن إسناده ل reference يشير لobject من نوع الأب ، لكن بالشكل التالي لا يمكن الوصول لل methods الموجودة بالابن، فقط يمكنك الوصول حصراً للتوابع التي في الأب والتوابع التي في الابن التي تكون overridden methods لتوابع موجودة بالأب.

و هذا المفهوم يدعى upcasting (سنستفيد منه أثناء القيام بما ندعوه dynamic binding بتمرين في آخر المحاضرة).

التمرين الثالث:

قم بتعريف 5 Classes :

■ :Animal Class

with method eat () : it prints "eating ..."

Then

■ Class dog which extends Animal class and override eat method to print "eating meat"

■ Class cat which extends Animal class and override eat method to print "eating abbitzs"

■ Class Lion which extends Animal class and override eat method to print "eating attles"

■ Then Main class :

فيه main method نقوم فيها بعمل reference من نوع Animal أي (Animal animal;) ثم نقوم كل مرة بإسناد كائن من كل الأنواع الثلاث السابقة له على التوالي، وفي كل مرة نسند كائناً من نوع نستدعي تابع eat التابع له.

File: Animal.java

```
1. package poly_concept
2. //Animal Class
3. public class Animal {
4.     public void eat(){
5.         System.out.println("eating.....");
6.     }}
```

File: Lion.java

```
1. package poly_concept
2. //Lion Class
3. public class Lion extends Animal{
4.     @Override
5.     public void eat(){
6.         System.out.println("eating cattles"); }}
```

File: Cat.java

```
1. package poly_concept
2. //Cat Class
3. public class Cat extends Animal{
4.     @Override
5.     public void eat(){
6.         System.out.println("eating rabbits"); }}
```

File: Dog.java

```
1. package poly_concept
2. //Dog Class
3. public class Dog extends Animal{
4.     @Override
5.     public void eat(){
6.         System.out.println("eating meat"); }}
```

File: Main.java

```
1. package poly_concept
2. //Main Class
3. public class Main {
4.     public static void main(String[] args) {
5.         Animal animal;
6.         animal = new Dog();
7.         animal.eat();
8.         animal = new Cat();
9.         animal.eat();
10.        animal = new Lion();
11.        animal.eat(); } }
```

تمارين الوظائف:

التمرين الأول:

File: Person.java

```

1. package inheritance_concept;
2. //Person Class
3. public class Person {
4.     //attributes
5.     String name;
6.     char gender;
7.     String birthday;
8.
9.     //parameterized constructor
10.    Person(String name , char gender , String birthday){
11.        this.name = name;
12.        this.gender = gender;
13.        this.birthday = birthday;
14.    }
15.
16.    //println method
17.    public void println(){
18.        System.out.println("The name is : " + name + "\n The
gender is : "+ gender + "\n The birthday is : "+ birthday);
19.    }
20. }

```

File: Student.java

```

1. package inheritance_concept;
2. //Student Class
3. public class Student extends Person {
4.     //attributes
5.     String university ;
6.     String college ;
7.     String section ;
8.
9.     //Parameterized constructor
10.    Student(String name , char gender , String birthday ,
String university ,String college , String section){
11.        //دائماً يجب أن نستدعي باني الأب ضمن باني الابن في حالة
لم يوجد باني افتراضي في الأب من خلال super
12.        super(name,gender,birthday);
13.

```

قم بتعريف 4 Classes

Person Class :

attributes :name -gender -
dateOfBirth

Methods : println and
Parameterized constructor

Employee Class :

يرث من Person

attributes: salary -
startingDate

methods: parameterized
constructor override the
println

Student Class :

يرث من Person

attributes: university -
college -section
methods: parameterized
constructor override the
println

Main Class :

فيه التابع main نقوم به
بتعريف instance (كائن) من
الطالب و من الموظف
وطباعة معلومات كليهما
باستخدام التابع println

```

14.     this.university=university;
15.     this.college = college;
16.     this.section = section;
17. }
18. @Override
19. public void printInfo() {
20.     // نستدعي نسخة التابع الموجودة بالأب التي
    تحوي طباعة معلومات عامة حول الشخص لتفادي التكرار
    من خلال super كذلك
21.     super.printInfo();
22.     // نطبع المعلومات الزائدة
23.     System.out.println("The student studies
at : " + university + " university at " + college + "
college at " + section + " section" );
24. }
25. }

```

File: Main.java

```

1. package inheritance_concept;
2. //تضمين المكتبة
3. import java.util.Scanner;
4. //Main Class
5. public class Main {
6.     public static void main(String[] args){
7.         //Scanner object
8.         Scanner scanner = new
Scanner(System.in);
9.         //إدخال بيانات الطالب
10.        System.out.println("starting with
student");
11.        System.out.println("enter the name of the
student");
12.        String name = scanner.nextLine();
13.        System.out.println("enter the gender
m/f");
14.        //لا يوجد طريقة صريحة لأخذ حرف
15.        //فبالتالي نأخذ نصاً ثم نستدعي تابعاً يأخذ حرفاً
ما من التابع و نطلب منه أن يأخذ الحرف الأول كما التالي
16.        // نأخذ الحرف الأول (0th index) من النص
الذي سيدخله المستخدم
17.        char gender = scanner.next().charAt(0);
18.        //لتفادي أخذ new line كما إدخال أسلفنا في
المحاضرة السابقة
19.        scanner.nextLine();

```

File: Employee.java

```

1. package inheritance_concept;
2. //Employee Class
3. public class Employee extends Person{
4.     //attributes
5.     double salary;
6.     String startDate;
7.     // parameterized constructor
8.     Employee(String name , char gender , String
birthday , double salary, String startDate){
9.         // دائماً يجب أن نستدعي باني الأب ضمن باني
الابن في حالة لم يوجد باني افتراضي في الأب من خلال
super
10.        super(name,gender,birthday);
11.        this.salary = salary;
12.        this.startDate = startDate;
13.    }
14.
15.    @Override
16.    public void printInfo() {
17.        // نستدعي نسخة التابع الموجودة بالأب التي
    تحوي طباعة معلومات عامة حول الشخص لتفادي التكرار
    من خلال super كذلك
18.        super.printInfo();
19.        // نطبع المعلومات الزائدة
20.        System.out.println("The salary is " +
salary + " and the employee started working in "
+ startDate);
21.    }
22. }

```

```

20. System.out.println("enter the birthday");
21. String birthday = scanner.nextLine();
22. System.out.println("enter the university
name");
23. String university = scanner.nextLine();
24. System.out.println("enter the college
name");
25. String college = scanner.next();
26. System.out.println("enter the section
name");
27. String section = scanner.next();
28. //تفادي أخذ new line كإدخال كما أسلفنا
في المحاضرة السابقة
29. scanner.nextLine();
30.
31. Student student = new
Student(name,gender,birthday,university,college,
section);
32.
33. // إدخال بيانات الموظف
34. System.out.println("employee now");
35. System.out.println("enter the name of
the employee");
36. name = scanner.nextLine();

```

```

37. System.out.println("enter the gender
m/f");
38. gender = scanner.next().charAt(0);
39. scanner.nextLine();
40.
41. System.out.println("enter the birthday");
42.
43. birthday = scanner.nextLine();
44. System.out.println("enter the start date
of your job");
45. String startDate = scanner.nextLine();
46. System.out.println("enter the salary");
47. double salary = scanner.nextDouble();
48. scanner.nextLine();
49.
50. Employee employee = new Employee
(name,gender,birthday,salary,startDate);
51. System.out.println("Student information:");
52. student.printInfo();
53. System.out.println("Employee
information:");
54. employee.printInfo();
55. }
56. }

```

التمرين الثاني:

عرف 3 classes:

- **Building Class:** buildingNumber, floors, floorArea, hasElevator and parameterized constructor and printInfo method
- **Hospital Class** which extends hospital: beds – departments and parameterized constructor and printInfo method
- **Main Class:** فيه التابع الرئيسي سنعرف خلاله مبنى عادي ومشفى ونطبع بياناتهما



File: Building.java

```

1. package inheritance_concept;
2. //Building Class
3. public class Building {
4.     //attributes
5.     private int buildingNumber;
6.     private int floors;
7.     private double floorArea;
8.     private boolean hasElevator;
9.
10.    //parameterized constructor
11.    public Building(int buildingNumber, int
floors, double floorArea, boolean hasElevator) {
12.        this.buildingNumber = buildingNumber;
13.        this.floors = floors;
14.        this.floorArea = floorArea;
15.        this.hasElevator = hasElevator;
16.    }
17.
18.    // Method to display building information
19.    public void displayInfo() {
20.        System.out.println("Building Number: " +
buildingNumber);
21.        System.out.println("Floors: " + floors);
22.        System.out.println("Floor Area: " +
floorArea);
23.        // راجع الملاحظة أدنى التمرين
24.        System.out.println("Has Elevator: " +
(hasElevator ? "Yes" : "No"));
25.    }
26. }

```

File: Hospital.java

```

1. package inheritance_concept;
2. //Hospital Class
3. public class Hospital extends Building {
4.     private int beds;
5.     private int departments;
6.
7.     //Parameterized constructor
8.     public Hospital(int buildingNumber, int
floors, double floorArea, boolean hasElevator, int
beds, int departments) {
9.         // استدعاء باني الأب
10.        super(buildingNumber, floors, floorArea,
hasElevator);
11.        this.beds = beds;
12.        this.departments = departments;
13.    }
14.
15.    // Override displayInfo to print more
information
16.    @Override
17.    public void displayInfo() {
18.        // نطبع معلومات المبنى العامة من خلال
استدعاء نسخة الأب من هذا التابع
19.        super.displayInfo();
20.        // طباعة باقي المعلومات
21.        System.out.println("Beds: " + beds);
22.        System.out.println("Departments: " +
departments);
23.    }
24. }

```

ملاحظة:

التعبير الشرطي: (hasElevator ? "Yes" : "No") هو اختصار لكتابة if-else

حيث يقوم بالتحقق من قيمة hasElevator ، في حال كانت قيمته true ستم طباعة Yes ، وفي حال كانت قيمته false ستم طباعة No.

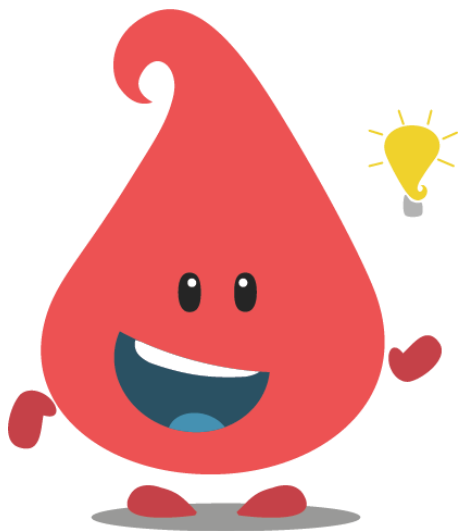
File: Main.java

```
1. package inheritance_concept;
2. //including libraries
3. import java.util.ArrayList;
4. import java.util.List;
5. import java.util.Scanner;
6.
7. //Main Class
8. public class Main {
9.     public static void main(String[] args) {
10.         //Scanner object
11.         Scanner scanner = new Scanner(System.in);
12.         //List of buildings
13.         List<Building> buildings = new ArrayList<>();
14.
15.         //عدد المباني المراد إدخالها
16.         System.out.print("Enter the number of buildings you want to add: ");
17.         int numberOfBuildings = scanner.nextInt();
18.         // حلقة للإدخال المباني
19.         for (int i = 0; i < numberOfBuildings; i++) {
20.             System.out.println("\n building " + (i + 1) + ":");
21.             //هل المبنى مشفى ؟
22.             System.out.print("Is this building a hospital? (yes/no): ");
23.             String isHospital = scanner.next();
24.
25.             //إدخالات مشتركة بين المباني
26.             System.out.print("Enter Building Number: ");
27.             int buildingNumber = scanner.nextInt();
28.             System.out.print("Enter Number of Floors: ");
29.             int floors = scanner.nextInt();
30.             System.out.print("Enter Floor Area: ");
31.             double floorArea = scanner.nextDouble();
32.             System.out.print("Does it have an Elevator? (true/false): ");
33.             boolean hasElevator = scanner.nextBoolean();
34.             // في حال كان المبنى مشفى (هذا التابع يقوم بمقارنة نص بنص مهماً كون الحرف كبيراً أم صغيراً)
35.             if (isHospital.equalsIgnoreCase("yes")) {
36.                 //إدخال بيانات المشفى
37.                 System.out.print("Enter Number of Beds: ");
38.                 int beds = scanner.nextInt();
39.                 System.out.print("Enter Number of Departments: ");
40.                 int departments = scanner.nextInt();
41.
42.             }
```

```

43.         buildings.add(new Hospital(buildingNumber, floors, floorArea,
hasElevator, beds, departments));
44.     }
45.     else // في حال لم يكن مشفى
46.     {
47.         // نقوم بإضافة مبنى جديد لـ arraylist المباني
48.         buildings.add(new Building(buildingNumber, floors, floorArea, hasElevator));
49.     }
50. }
51.
52. // طباعة معلومات المباني
53. System.out.println("\nBuildings Information:");
54. // foreach للمرور على كل العناصر ضمن الـ arraylist
55. // راجع الملاحظة أدنى التمرين
56. for (Building building : buildings) {
57.     building.displayInfo();
58.     System.out.println("-----");
59. }
60. }
61. }

```



ملاحظة: بما أن الـ object من نوع Hospital هو كذلك object من نوع Building فإنه يمكن وضع أي object من نوع Hospital في أي container من النوع Building.

الـ container: مكان لتخزين الكائنات، من أمثله الـ ArrayList. في حالتنا كان لدينا arraylist من النوع Building يمكننا أن نحوي objects من النوع building وكذلك من النوع hospital.

و عند محاولة استخدام تابع موجود بالأب تم عمل override له ضمن الابن من خلال object موجود في هذه الـ arraylist (في حالتنا displayInfo) فإن java قادرة على معرفة هل الـ object المشار إليه من نوع building أم من نوع hospital و استدعاء النسخة المناسبة من التابع تبعاً لنوع هذا الكائن.

و نسمي هذه العملية dynamic binding ، حيث تقوم جافا باكتشاف نوع الـ object الفعلي في الـ container بدلاً من النظر فقط للـ reference الذي هو من نوع الحاوية أو الـ container التي نستخدمها للتخزين.

المقصود ب reference هنا: أي object سيتم إضافته ل arraylist من هذا النوع.
مثلاً:

```
List<Building> buildings = new ArrayList<>();
```

سيتعرف ضمناً على الشكل التالي:

```
Building building = new (Building(.....)/Hospital(.....)) *
```

فكافة objects هذه ال container تعرف على نفس النحو.
فال dynamic binding ينظر لنوع الأوبجكت الفعلي الذي أنشأناه بناءً عليه ، أي لو أنشأنا Hospital في * سيتم اكتشاف أنه hospital بغض النظر عن كوننا نملك reference من نوع Building بالمقام الأول نشير لهذا ال object باستخدامه (قمنا في التمرين بتطبيق مفهوم ال Upcasting السابق شرحه).

حظاً طيباً يا أصدقاء

